

API Documentation

Routing, Validation, Authentication, Authorization, Controllers and Responses

1. Overview

This document describes the current API implementation, including:

- Express application configuration
 - Rate limiting
 - Authentication and authorization middleware placement
 - API routes
 - Request validation
 - User management
 - API registration and login
 - Friendship management
 - Inventory management
 - Error handling
 - HTTP response codes
 - Security considerations
 - Implementation observations and recommended improvements
-

2. Application Configuration

The API is built using Express.

The application enables support for URL-encoded and JSON request bodies:

```
const app = express();
```

```
app.set("trust proxy", 1);
```

```
app.use(urlencoded({ extended: true }));
```

```
app.use(json());
```

2.1 Proxy Configuration

The application uses:

```
app.set("trust proxy", 1);
```

This configures Express to trust the first proxy in front of the application.

This is commonly required when the application is deployed behind a reverse proxy or load balancer, particularly when middleware such as rate limiting relies on the client's IP address.

The proxy configuration should match the actual deployment architecture. Trusting proxies incorrectly can result in inaccurate client IP detection.

3. Rate Limiting

The API uses two separate rate limiters.

3.1 Public Rate Limiter

Configuration

Property	Value
Time window	60 seconds
Maximum requests	20
HTTP status when exceeded	429

The limiter is applied to:

POST /api/register

POST /api/login

DELETE /api/quit

Rate-limit response

HTTP/1.1 429 Too Many Requests

Too many requests. Please try again later.

3.2 Authenticated API Rate Limiter

Configuration

Property	Value
Time window	60 seconds
Maximum requests	120
Administrator exemption	Yes
HTTP status when exceeded	429

Administrators are excluded from this limiter

4. Application-Level Middleware Order

Client Request

|



Express Body Parsing

|



Route Validation

|



Public Rate Limiter

|



Authentication

|



API Rate Limiter

|



Role Authorization

|



Controller

|



Database

|



Response

The exact middleware order for individual router routes depends on how the router is mounted and how the route-level middleware is configured.

5. Authentication and Authorization

Protected endpoints use:

`authMiddleware`

Administrative endpoints additionally use:

`roleMiddleware("admin")`

Authentication determines whether the request contains valid credentials.

Authorization determines whether the authenticated user has permission to perform the requested operation.

The general structure is:

Authentication

|

▼

Is the user authenticated?

|

|— No → 401 Unauthorized

|

▼

Authorization

|

▼

Does the user have the required role?

|

|— No → 403 Forbidden

|

▼

Controller

6. API Route Structure

The API exposes routes for:

- API registration
- Login
- API-user deletion
- User management
- Friendship management
- Inventory management

The main router routes are described below.

7. Authentication Routes

Method	Endpoint	Authentication	Admin
POST	/api/register	No	No
POST	/api/login	No	No
DELETE	/api/quit	No	No

These endpoints use the public rate limiter.

8. User Routes

Method	Endpoint	Authentication	Admin
GET	/get-all-users	Yes	No
GET	/user	Yes	No
GET	/admin/user	Yes	Yes
POST	/user	Yes	Yes
PATCH	/user/:id	Yes	Yes
PUT	/user/:id	Yes	Yes
DELETE	/user/:id	Yes	Yes

9. Friendship Routes

Method	Endpoint	Authentication	Admin
GET	/friends/:id	Yes	Yes
POST	/friends/	Yes	Yes
DELETE	/friends/	Yes	Yes

10. Inventory Routes

Method	Endpoint	Authentication	Admin
PATCH	/inventory/add	Yes	Yes
PATCH	/inventory/remove	Yes	Yes

11. Validation

The API uses validation middleware before controllers are executed.

Validation is used to check:

- usernames
- email addresses
- passwords
- numeric IDs
- friendship users
- friendship acceptance state
- inventory users
- inventory items
- inventory ownership

The validation layer also performs normalization and sanitization for several fields.

12. Username Validation

Required Username

A required username must:

1. Exist in the request.
2. Be a string.
3. Be trimmed.
4. Contain between 4 and 128 characters.

The effective validation rule is:

`^.{4,128}$`

Invalid values produce:

Error: Invalid username

A missing username can produce:

Error: Username not in request

13. Email Validation

Required email validation performs the following operations:

1. Checks that the field exists.
2. Verifies that it is a string.
3. Trims whitespace.
4. Escapes the value.
5. Validates the email format.
6. Normalizes the email.
7. Applies the following regular expression:

```
^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$
```

Possible errors include:

Error: Email not in request

and:

Error: Invalid Email

14. Password Validation

Passwords must satisfy all of the following requirements:

- Minimum length: 12 characters
- Maximum length: 67 characters
- At least one lowercase letter
- At least one uppercase letter
- At least one number
- At least one special character
- No whitespace

The validation expression is:

```
^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[^A-Za-z0-9])[^\\s]{12,67}$
```

Possible errors include:

Error: Password not in request

and:

Error: Invalid password

15. Optional User Update Validation

The PATCH user-update endpoint allows update fields to be optional.

For example:

```
{  
  "username": "newusername"  
}
```

or:

```
{  
  "email": "new@example.com"  
}
```

or:

```
{  
  "password": "NewSecurePassword123!"  
}
```

When supplied, the fields are validated according to their respective rules.

16. User ID Validation

ID parameters are converted to integers and validated using:

`toInt()`

`isInt()`

For example:

`DELETE /user/15`

The same principle is used for inventory IDs.

17. Friendship Validation

Friendship requests validate:

```
{
  "user1": 1,
  "user2": 2,
  "isaccepted": true
}
```

user1 and user2 are expected to be numeric values.

isaccepted is converted to a Boolean and validated as a Boolean.

18. Inventory Validation

Inventory requests validate:

```
{
  "user": 1,
  "item": 3
}
```

Both values are converted to integers and checked as valid integers.

The inventory ownership field is also converted to a Boolean where applicable.

19. Centralized Validation Error Handling

The API uses `HandleParsingError()` to convert validation-library errors into API-specific messages.

For username, email, and password errors, the helper can return:

Error: Username not in request

Error: Invalid username

Error: Email not in request

Error: Invalid Email

Error: Password not in request

Error: Invalid password

For other validation failures, the validation result is serialized and passed to the error handler.

Example:

```
[{"type": "field", "msg": "Invalid  
value", "path": "user1", "location": "body"}]
```

20. API Registration

Endpoint

POST /api/register

Request Body

```
{  
  "email": "user@example.com",  
  "password": "ExamplePassword123!"  
}
```

Purpose:

1. Searches for an existing user by email.
2. Verifies the supplied password.
3. Checks whether the user is already registered for API access.
4. Generates an API secret.
5. Creates an API-user record.

Therefore, this endpoint is more accurately described as:

Register an existing application user for API access.

Successful Response

HTTP 201 — Created

```
{  
  "Data": [  
    {  
      "user": 1,  
      "role": "user",  
      "id": 1  
    }  
  ]  
}
```

The exact returned fields depend on the database schema.

The API secret is removed before returning the response:

```
delete Data[0].secret_key;
```

Registration Errors

User Does Not Exist

400 Bad Request

Error: Api register wrong credentials

Incorrect Password

400 Bad Request

Error: Api register wrong credentials

Already Registered

400 Bad Request

Error: user already register to the api

Internal Error

500 Internal Server Error

Error: Failed to add api user

21. Login

Endpoint

POST /api/login

Request Body

```
{  
  "email": "user@example.com",  
  "password": "ExamplePassword123!"  
}
```

Authentication Process

The login controller:

1. Validates the request.
2. Finds the user by email.

3. Compares the supplied password using bcrypt.
4. Retrieves the user's API role and secret.
5. Generates a JWT.
6. Returns the token and role information.

The JWT uses:

HS256

and includes:

```
{  
  "id": 1,  
  "role": "user"  
}
```

The token also has an expiration of 1h configured by:

`expire_time`

22. Login Success Response

HTTP 201 — Created

The current implementation returns:

```
{  
  "token": "JWT_TOKEN",  
  "message": "Logged in successfully",  
  "role": [  
    {  
      "role": "user",  
    }  
  ],  
}
```

23. Login Errors

Invalid User

400 Bad Request

Error: Api register wrong credentials

Incorrect Password

400 Bad Request

Error: Api register wrong credentials

Validation Error

400 Bad Request

Possible messages:

Error: Email not in request

Error: Invalid Email

Error: Password not in request

Error: Invalid password

Internal Error

500 Internal Server Error

Error: Failed to add api user

24. Delete API User

Endpoint

DELETE /api/quit

Request Body

```
{  
  "email": "user@example.com",  
  "password": "ExamplePassword123!"  
}
```

Successful Response

HTTP 200 — OK

```
{  
  "msg": "User delete from the data base"  
}
```

Errors

User Does Not Exist

400 Bad Request

Error: Api register wrong credentials

Incorrect Password

400 Bad Request

Error: Api wrong password

Not Registered for API Access

400 Bad Request

Error: user not register in api

Internal Error

500 Internal Server Error

Error: Failed to delete user

25. Get All Users

Endpoint

GET /get-all-users

Authentication

Authentication required.

Administrator authorization is not explicitly applied by this route.

Successful Response

HTTP 200 — OK

The controller returns all users while removing sensitive fields such as:

- password
- email

Example:

```
{
  "Data": [
    {
      "id": 1,
      "username": "exampleuser"
    },
    {
      "id": 2,
      "username": "anotheruser"
    }
  ]
}
```

The exact fields depend on the database schema.

Internal Error

HTTP 500 — Internal Server Error

Error: Error: Failed to fetch all Users

26. Get User

Endpoint

GET /user

Request Body

```
{
  "username": "exampleuser"
}
```

Authentication

Authentication required.

Successful Response

HTTP 200 — OK

```
{
  "Data": [
    {
      "id": 1,
      "username": "exampleuser"
    }
  ]
}
```

Password and email are removed from the returned object.

User Not Found

HTTP 404 — Not Found

Error: User not found

Validation Error

HTTP 400 — Bad Request

Error: Invalid username

or:

Error: Username not in request

Internal Error

HTTP 500 — Internal Server Error

Error: Failed to fetch user

27. Admin Get User

Endpoint

GET /admin/user

Request Body

```
{
  "username": "exampleuser"
}
```

Authentication

Authentication and administrator authorization are required.

Successful Response

HTTP 200 — OK

The controller removes the user's password before returning the data.

Unlike the normal `/user` endpoint, the current implementation does not remove the email address.

Example:

```
{
  "Data": [
    {
      "id": 1,
      "username": "exampleuser",
      "email": "user@example.com"
    }
  ]
}
```

User Not Found

404 Not Found

Error: User not found

Internal Error

500 Internal Server Error

Error: Failed to fetch user

28. Add User

Endpoint

POST `/user`

Authentication

Authentication and administrator authorization are required.

Request Body

```
{
```

```
"username": "exampleuser",  
"email": "user@example.com",  
"password": "ExamplePassword123!"  
}
```

Process

The controller:

1. Validates the username.
2. Validates the email.
3. Validates the password.
4. Checks whether the user already exists.
5. Hashes the password using bcrypt.
6. Creates the user.
7. Retrieves shop products.
8. Creates inventory records for the user.
9. Removes the password from the response.
10. Returns the created user.

The password is hashed with a bcrypt cost factor of:

10

Successful Response

HTTP 201 — Created

```
{  
  "Data": [  
    {  
      "id": 1,  
      "username": "exampleuser",  
      "email": "user@example.com"  
    }  
  ]  
}
```

User Already Exists

HTTP 400 — Bad Request

Error: user already in database

Database Errors

Possible responses include:

Error: email already exist

Error: Username already exist

Error: User not found in the data base

Internal Error

HTTP 500 — Internal Server Error

Error: Failed to add user

29. Update User — PATCH

Endpoint

PATCH /user/:id

Authentication

Authentication and administrator authorization are required.

Example

PATCH /user/1

Request Body

Any supported update fields can be supplied.

Example:

```
{  
  "username": "newusername",  
  "email": "new@example.com"  
}
```

or:

```
{  
  "password": "NewSecurePassword123!"  
}
```

Successful Response

HTTP 200 — OK

```
{
  "Data": [
    {
      "id": 1,
      "username": "newusername",
      "email": "new@example.com"
    }
  ]
}
```

User Not Found

HTTP 404 — Not Found

Error: User not found

Database Errors

Possible duplicate-field errors include:

Error: Username already exist

or:

Error: email already exist

Internal Error

HTTP 500 — Internal Server Error

Error: Failed to update user

30. Update User — PUT

Endpoint

PUT /user/:id

Authentication

Authentication and administrator authorization are required.

Example

PUT /user/1

Request Body

```
{  
  "username": "newusername",  
  "email": "new@example.com",  
  "password": "NewSecurePassword123!"  
}
```

Successful Response

HTTP 201 — Created

```
{  
  "Data": [  
    {  
      "id": 1,  
      "username": "newusername",  
      "email": "new@example.com"  
    }  
  ]  
}
```

User Not Found

HTTP 404 — Not Found

Error: User not found

Internal Error

HTTP 500 — Internal Server Error

Error: Failed to update user

31. Delete User

Endpoint

DELETE /user/:id

Authentication

Authentication and administrator authorization are required.

Example

DELETE /user/1

Process

The controller:

1. Finds the user.
2. Retrieves the available shop products.
3. Removes the user's inventory records.
4. Deletes the user.
5. Returns the deleted user's ID.

Successful Response

HTTP 200 — OK

```
{
  "Data": [
    {
      "deleteDataId": 1
    }
  ]
}
```

User Not Found

HTTP 404 — Not Found

Error: User not found

Internal Error

HTTP 500 — Internal Server Error

Error: Failed to delete user

32. Add Friendship

Endpoint

POST /friends/

Authentication

Authentication and administrator authorization are required.

Request Body

```
{  
  "user1": 1,  
  "user2": 2,  
  "isaccepted": true  
}
```

Successful Response

HTTP 201 — Created

```
{  
  "Data": [  
    {  
      "user1": 1,  
      "user2": 2,  
      "isaccepted": true  
    }  
  ]  
}
```

Validation Error

HTTP 400 — Bad Request

The validation errors are serialized.

Example:

```
[{"type": "field", "msg": "Invalid  
value", "path": "user1", "location": "body"}]
```

Database Errors

Possible responses include:

Error: user not existing in database

Error: No valid field in request

Error: email already exist

Error: Username already exist

Error: User not found in the data base

Internal Error

HTTP 500 — Internal Server Error

Error: Failed to add friends

33. Delete Friendship

Endpoint

DELETE /friends/

Authentication

Authentication and administrator authorization are required.

Request Body

```
{
  "user1": 1,
  "user2": 2
}
```

Successful Response

HTTP 200 — OK

```
{
  "Data": [
    {
      "User1": 1,
      "User2": 2
    }
  ]
}
```

Friendship Not Found

HTTP 404 — Not Found

Error: Users not found or not in friends list

Validation Error

HTTP 400 — Bad Request

The serialized validation errors are returned.

Internal Error

HTTP 500 — Internal Server Error

Error: Failed to delete Friend link

34. Get Friends

Endpoint

GET /friends/:id

Authentication

Authentication and administrator authorization are required.

Example

GET /friends/1

Successful Response

HTTP 200 — OK

```
{
  "Data": [
    {
      "user1": 1,
      "user2": 2,
      "isaccepted": true
    },
    {
      "user1": 1,
      "user2": 5,
      "isaccepted": false
    }
  ]
}
```

Validation Error

HTTP 400 — Bad Request

An invalid ID can result in serialized validation errors.

No Friendship Records

HTTP 400 — Bad Request

Error: user not found

The current controller returns this response when no friendship records are found.

This does **not necessarily mean that the user does not exist**. It can simply mean that the user has no matching friendship records.

Internal Error

HTTP 500 — Internal Server Error

Error: Failed to fetch friends

35. Add Inventory Item

Endpoint

PATCH /inventory/add

Authentication

Authentication and administrator authorization are required.

Request Body

```
{  
  "user": 1,  
  "item": 3  
}
```

Validation

The controller checks:

- user exists in the request
- item exists in the request
- user is a valid integer
- item is a valid integer
- item ID is not greater than 8

Successful Response

HTTP 201 — Created

"user: 1 now own item 3"

Missing User

HTTP 400 — Bad Request

Error: User not in request

Missing Item

HTTP 400 — Bad Request

Error: Item not in request

Invalid Item

HTTP 400 — Bad Request

Error: No valid id for item

The current implementation rejects item IDs greater than 8.

Already Owns Item

HTTP 400 — Bad Request

Error: user already own the item

User Not Found

HTTP 400 — Bad Request

Error: user not existing in database

Internal Error

HTTP 500 — Internal Server Error

Error: failed to add the item

36. Remove Inventory Item

Endpoint

PATCH /inventory/remove

Authentication

Authentication and administrator authorization are required.

Request Body

```
{  
  "user": 1,  
  "item": 3  
}
```

Successful Response

HTTP 201 — Created

"user: 1 now don't own item 3"

Missing User

HTTP 400 — Bad Request

Error: User not in request

Missing Item

HTTP 400 — Bad Request

Error: Item not in request

Invalid Item

HTTP 400 — Bad Request

Error: No valid id for item

Item Not Owned

HTTP 400 — Bad Request

Error: user already don't own the item

User Not Found

HTTP 400 — Bad Request

Error: user not existing in database

Internal Error

HTTP 500 — Internal Server Error

37. Database Error Handling

The API uses `handleErrorCode()` to translate selected database errors into API responses.

PostgreSQL 42601

The API returns:

Error: No valid field in request

with:

400 Bad Request

PostgreSQL 23505

A unique-constraint violation can result in:

Error: email already exist

or:

Error: Username already exist

with:

400 Bad Request

PostgreSQL 23503

A foreign-key violation results in:

Error: User not found in the data base

with:

404 Not Found

39. Important Error-Handling Behavior

The helper contains:

```
if (!used) {
    next(
        new CustomError(
            "user not existing in database",
            400
        )
    );

    return 1;
}
```

This means that when a controller calls:

```
handleErrorCode(error, next, null)
```

the helper immediately produces:

Error: user not existing in database

with status 400.

39. Response Status Codes

The API currently uses the following status codes.

Status	Meaning
200	Successful operation
201	Resource/state created or modified
400	Invalid input or business-rule failure
401	Authentication failure
403	Insufficient permissions
404	Resource not found
429	Rate limit exceeded
500	Unexpected server error

40. Endpoint Response Summary

Method	Endpoint	Success
POST	/api/register	201
POST	/api/login	201
DELETE	/api/quit	200
GET	/get-all-users	200
GET	/user	200
GET	/admin/user	200
POST	/user	201
PATCH	/user/:id	200
PUT	/user/:id	201
DELETE	/user/:id	200
GET	/friends/:id	200
POST	/friends/	201
DELETE	/friends/	200
PATCH	/inventory/add	201
PATCH	/inventory/remove	201

41. Security Architecture

The API uses multiple layers of protection.

Client Request



Express Parsing



Validation



Rate Limiting

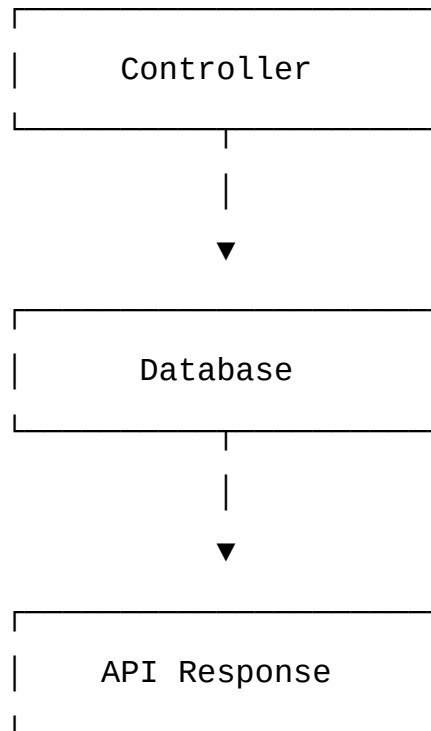


Authentication



Authorization





This layered architecture helps protect the application against:

- malformed input
- unauthorized access
- unauthorized administrative operations
- excessive requests
- accidental exposure of database passwords

42. Password Security

Passwords are processed using bcrypt.

Passwords are removed from response objects before returning user information.

43. JWT Security

The login controller creates a JWT using the HS256 algorithm.

The token contains:

```
{
  "id": 1,
  "role": "user"
}
```

The token has an expiration period.

The JWT signing secret should remain private.

56. Complete API Endpoint Reference

Method	Endpoint	Purpose	Auth	Admin
POST	/api/register	Register existing user for API access	No	No
POST	/api/login	Authenticate and receive JWT	No	No
DELETE	/api/quit	Remove API access	No	No
GET	/get-all-users	Retrieve users	Yes	No
GET	/user	Retrieve a user	Yes	No
GET	/admin/user	Retrieve a user as administrator	Yes	Yes
POST	/user	Create a user	Yes	Yes
PATCH	/user/:id	Partially update a user	Yes	Yes
PUT	/user/:id	Replace/update a user	Yes	Yes
DELETE	/user/:id	Delete a user	Yes	Yes
GET	/friends/:id	Retrieve friendship records	Yes	Yes
POST	/friends/	Create a friendship	Yes	Yes
DELETE	/friends/	Delete a friendship	Yes	Yes
PATCH	/inventory/add	Add an inventory item	Yes	Yes
PATCH	/inventory/remove	Remove an inventory item	Yes	Yes

57. Complete Error Reference

HTTP	Error
400 Error:	Username not in request
400 Error:	Invalid username
400 Error:	Email not in request
400 Error:	Invalid Email
400 Error:	Password not in request
400 Error:	Invalid password
400 Error:	Api register wrong credentials
400 Error:	Api wrong password
400 Error:	user already register to the api
400 Error:	user not register in api
400 Error:	user already in database
400 Error:	User not in request
400 Error:	Item not in request
400 Error:	No valid id for item
400 Error:	user already own the item
400 Error:	user already don't own the item

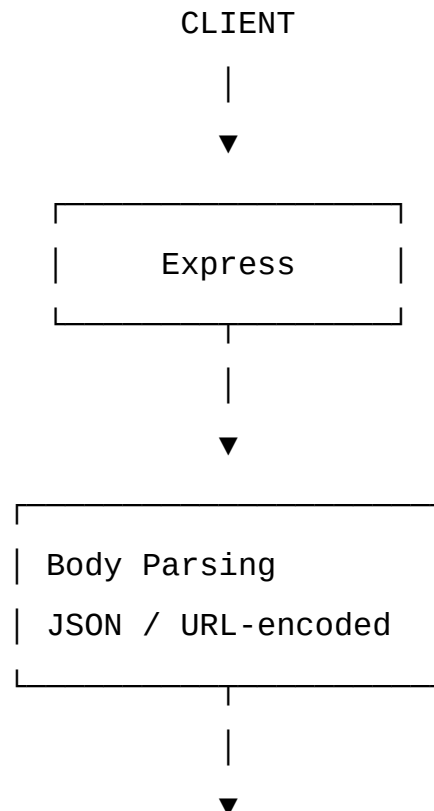
HTTP

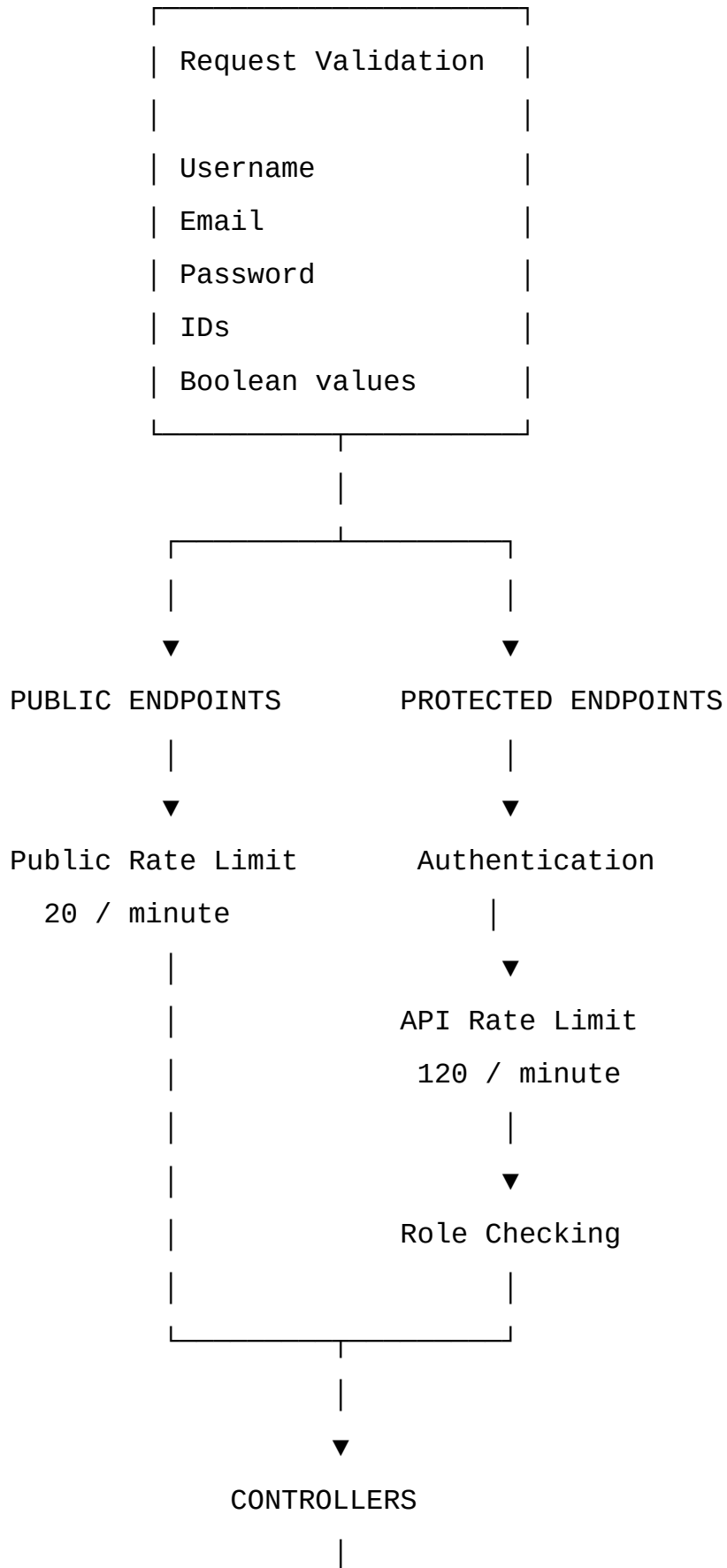
Error

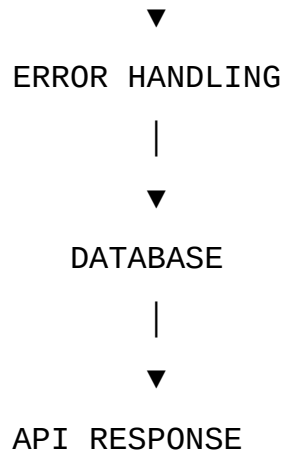
400 Error: user not existing in database
400 Error: No valid field in request
400 Error: user not found
404 Error: User not found
404 Error: Users not found or not in friends list
404 Error: User not found in the data base
429 Too many requests. Please try again later.
500 Error: Failed to add api user
500 Error: Failed to delete user
500 Error: Failed to fetch user
500 Error: Failed to fetch friends
500 Error: Failed to delete user
500 Error: Failed to add friends
500 Error: Failed to delete Friend link
500 Error: failed to add the item
500 Error: Failed to update user

59. Final Architecture

The complete API architecture can be summarized as:







60. Conclusion

The API uses a layered Express architecture combining request parsing, validation, rate limiting, authentication, authorization, database operations, and centralized error handling.

The current implementation provides endpoints for:

- API authentication
- User management
- Friendship management
- Inventory management

It also implements password hashing, JWT authentication, role-based authorization, and request-rate limiting.

Finally, the API would benefit from standardized response formats, consistent error messages, and more conventional HTTP status codes.

The recommended long-term architecture is:

Validate



Rate Limit



Authenticate



Authorize



Execute Controller



Validate Database Result



Return Consistent Response



Log Only Non-Sensitive Information